

Guía de Deployment - EMA200 App

Opción 1: Deployment Local (Desarrollo)

Requisitos

- Node.js 18+
- PostgreSQL 12+
- npm o yarn

Pasos

1. Clonar y preparar

```
cd ema200-jubilacion
```

2. Configurar Base de Datos

```
# Crear base de datos
createdb ema200_jubilacion

# O si usas psql:
psql -U postgres
# \create database ema200_jubilacion;
# \q
```

3. Inicializar Backend

```
cd backend

# Las dependencias ya están instaladas
# npm install

# Inicializar base de datos
npm run db:init

# Iniciar en modo desarrollo
npm run dev
```

Backend ejecutándose en `http://localhost:5000`

4. Iniciar Frontend

```
cd ../frontend

# Las dependencias ya están instaladas
# npm install

# Iniciar servidor de desarrollo
npm run dev
```

Frontend ejecutándose en `http://localhost:5173`

Opción 2: Deployment con Docker

Requisitos

- Docker
- Docker Compose

Pasos

```
# Construir y ejecutar contenedores
docker-compose up -d

# Ver logs
docker-compose logs -f

# Detener
docker-compose down
```

URLs:

- Frontend: `http://localhost:3000`
- Backend: `http://localhost:5000`
- PostgreSQL: `localhost:5432`

Opción 3: Deployment en Railway

Pasos

1. **Crear cuenta en Railway:** <https://railway.app>
2. **Conectar repositorio Git**
3. **Configurar variables de entorno:**

```
DATABASE_URL=postgresql://[user]:[password]@[host]:[port]/ema200_jubilacion
NODE_ENV=production
FRONTEND_URL=https://[tu-app].up.railway.app
TWELVE_DATA_API_KEY=37df2fa94b664d9b969beb200222fcaf
BREVO_API_KEY=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
JWT_SECRET=[generar-valor-aleatorio-fuerte]
```

1. **Crear servicios:**
 - PostgreSQL (desde Railway services)
 - Backend (Node.js)
 - Frontend (Static)
2. **Deploy automático** en cada push a main

Opción 4: Deployment en Render

Pasos

1. **Crear cuenta en Render:** <https://render.com>

2. Crear servicio PostgreSQL:

- Type: PostgreSQL
- Guardar connection string

3. Crear servicio Backend:

- Type: Web Service
- Build command: `npm install`
- Start command: `npm start`
- Variables de entorno:

```
env
```

```
DATABASE_URL=[de PostgreSQL]
```

```
NODE_ENV=production
```

```
PORT=10000
```

4. Crear servicio Frontend:

- Type: Static Site
- Build command: `npm install && npm run build`
- Publish directory: `dist`
- Environment variable:

```
env
```

```
VITE_API_URL=https://[tu-backend].onrender.com/api
```

5. Deploy desde el panel de Render

Opción 5: Deployment en Heroku (Deprecated pero aún funciona)

Pasos

```
# Instalar Heroku CLI
npm install -g heroku

# Login
heroku login

# Crear apps
heroku create ema200-backend
heroku create ema200-frontend

# Configurar add-ons (PostgreSQL)
heroku addons:create heroku-postgresql:hobby-dev -a ema200-backend

# Configurar variables de entorno
heroku config:set \
  NODE_ENV=production \
  FRONTEND_URL=https://ema200-frontend.herokuapp.com \
  TWELVE_DATA_API_KEY=37df2fa94b664d9b969beeb200222fcac \
  BREVO_API_KEY=xkeysib-... \
  JWT_SECRET=... \
  -a ema200-backend

# Deploy
git push heroku main
```

Opciones de Hosting de Base de Datos

AWS RDS

```
Host: [tu-host].rds.amazonaws.com
Port: 5432
Database: ema200_jubilacion
Username: postgres
Password: [tu-contraseña]
```

ElephantSQL (PostgreSQL as a Service)

- Servicio: <https://www.elephantsql.com/>
- Plan gratuito disponible
- Connection string automática

Railway PostgreSQL

- Incluido en Railway
- Backup automático
- Escalable

Consideraciones de Producción

Seguridad

- ☐ Cambiar JWT_SECRET por valor fuerte aleatorio
- ☐ Habilitar HTTPS
- ☐ Configurar CORS correctamente
- ☐ Rate limiting en APIs
- ☐ Validación de inputs stricter
- ☐ Usar variables de entorno para secretos

Performance

- ☐ Habilitar caching en frontend
- ☐ Compresión GZIP en servidor
- ☐ CDN para assets estáticos
- ☐ Lazy loading de componentes React
- ☐ Índices en base de datos

Monitoreo

- ☐ Logging centralizado
- ☐ Error tracking (Sentry)
- ☐ Uptime monitoring
- ☐ Alertas en cron jobs fallidos

Backups

- ☐ Backup diario de BD
- ☐ Versioning de código
- ☐ Plan de recuperación ante desastres

Variables de Entorno Requeridas

Backend:

```
# Base de datos
DB_HOST=localhost
DB_PORT=5432
DB_NAME=ema200_jubilacion
DB_USER=postgres
DB_PASSWORD=postgres

# Server
NODE_ENV=production
PORT=5000
FRONTEND_URL=https://app.ejemplo.com

# Security
JWT_SECRET=tu-secret-muy-largo-y-aleatorio

# Timezone
TZ=Europe/Madrid
```

Frontend:

```
VITE_API_URL=https://api.ejemplo.com/api
```

Troubleshooting

Error: ECONNREFUSED en Backend

- Verificar que PostgreSQL está ejecutándose
- Verificar credenciales de BD
- Verificar host y puerto

Error: CORS en Frontend

- Verificar que FRONTEND_URL en backend es correcto
- Revisar headers CORS en backend

Cron jobs no se ejecutan

- Verificar timezone en servidor
- Revisar logs de servidor
- Considerar usar servicio externo de cron (easycron, cron-job.org)

Emails no se envían

- Verificar API key de Brevo
- Revisar plan de Brevo (límites de emails)
- Revisar logs de backend
- Validar direcciones de email

Monitoreo de Cron Jobs

Para verificar que los cron jobs están funcionando:

```
# Ver logs del backend
tail -f logs/backend.log | grep -i cron

# Consultar historial EMA200 en BD
SELECT * FROM ema200_history ORDER BY date DESC LIMIT 10;

# Consultar emails enviados
SELECT * FROM email_log ORDER BY sent_at DESC LIMIT 20;
```

Escalabilidad Futura

Si la aplicación crece:

- Separar cron jobs en worker separado
- Usar Redis para caché
- Implementar queue system (Bull, RabbitMQ)
- Escalar base de datos (replicación, sharding)
- CDN para frontend estático
- Microservicios para funcionalidades específicas